Group No: 25

Problem statement: DEMAND FORECASTING & STOCK OPTIMISATION FOR FAST-MOVING PRODUCTS

1. Identification & Justification of Process Model

Selected Model: Agile Process Model

1.1 Model Overview

For this AI/ML supply chain project, we have selected the Agile process model. Agile focuses on building software in small, manageable cycles (sprints) rather than trying to build everything at once. This is the best fit because machine learning is a trial-and-error process. We need to continuously test the model with data, check the results, and improve it step-by-step alongside building the software dashboard.

1.2 Justification & Relevance (Why Agile?)

We chose Agile because traditional models (like Waterfall) are too rigid for AI development. Agile provides the flexibility we need to achieve our specific project outcomes:

1. Iterative Model Building for Better Accuracy AI models are rarely perfect on the first try. Agile allows us to build a basic version first to gather **Data-Driven Insights** about demand patterns. Once the base model is working, we can use the next sprint to improve its accuracy. This step-by-step improvement directly helps us **Reduce Stockouts** by making sure our predictions get sharper over time, ensuring products are available when needed.

2. Step-by-Step Feature Integration Agile lets us build the software in distinct modules. We can first deliver the core model to **Minimise Wastage** by optimizing inventory levels. In the next sprint, we can add the UI for **Real-Time Monitoring** to track inventory dynamically. Finally, we can add notification systems to provide **Actionable Alerts** to supply chain managers. This prevents us from getting stuck trying to build everything at once.

3. Focusing on Immediate Business Value Agile prioritizes delivering the most important features first. By quickly deploying a working system that balances stock levels, we can immediately show how the software helps **Improve Working Capital Efficiency** and balance cash flow, which is the core business goal of the project.

4. Adaptability and Scalability In Agile, changes are expected and welcome. If we realize during testing that our data needs to handle a completely different type of product, Agile allows us to easily adjust our software design in the next cycle. This guarantees a **Scalable Solution** that is adaptable across different industries and product categories without having to restart the entire project from scratch.

1.3 Rejected Process Models

1. Waterfall Model

- **Why we rejected it:** Waterfall is strictly linear—you have to finish one phase completely before moving to the next, and you cannot easily go back. Machine learning is a trial-and-error process. If our model gives poor predictions during the testing phase, we *have* to go back and change the data or the algorithm. Waterfall is simply too rigid for AI development.

2. Spiral Model

- **Why we rejected it:** The Spiral model is built for massive, multi-year, high-budget projects (like aerospace software) because it spends a huge amount of time on deep risk analysis. For a fast-paced hackathon, this is overkill. It would slow down our actual coding and development time unnecessarily.

3. Iterative Model

- **Why we rejected it:** While the Iterative model builds software in chunks, it still relies on having most of the main requirements locked in early. It lacks the fast, flexible feedback loops of Agile. Agile allows us to instantly pivot and add new features (like real-time alerts) based on what the data shows us, which standard iterative models don't handle as smoothly.

4. Prototyping Model

- **Why we rejected it:** This model usually involves building a "dummy" or throwaway version of the software just to figure out what the user wants, and then building the real thing from scratch. In a hackathon, we don't have time to throw work away. Our initial ML experiments need to evolve directly into the final working product, making Agile a much better fit.

Criteria	Agile ✓	Waterfall ✗	Spiral	RAD
Handles changing requirements	✓ Excellent	✘ Poor	✓ Good	✓ Good
Suitable for ML/AI projects	✓ Best fit	✘ No	✓ Partial	✘ Limited
Early working software	✓ Yes	✘ Late	✘ Late	✓ Yes
Stakeholder involvement	✓ Continuous	✘ Minimal	✓ Regular	✓ High
Risk management	✓ Per sprint	✘ End-stage	✓ Built-in	✘ Moderate
Scalable architecture	✓ Iterative	✘ One-shot	✓ Planned	✘ Limited
Testing approach	✓ Every sprint	✘ Final phase	✓ Phase-wise	✓ Module-wise

2. Problem Statement

Inaccurate demand forecasting causes stockouts and overstocking.

Businesses face inventory wastage and financial losses.

Existing systems lack intelligent forecasting and proactive alerts.

The project aims to improve supply chain decisions using AI/ML.

3. Requirement Gathering

Requirement Gathering Techniques

Stakeholder Analysis

Domain Research

Dataset Analysis

Market Research

3.1. Stakeholders

Supply Chain Manager

Warehouse Manager

Admin

Retail Business Owner

3.2 Problems Identified

Stockouts

Overstocking

Delayed inventory decisions

Lack of analytics

Inefficient reorder planning

4. Objectives

Predict product demand accurately

Optimize inventory levels

Reduce wastage

Generate intelligent alerts

Improve supply chain efficiency

Provide analytics dashboard